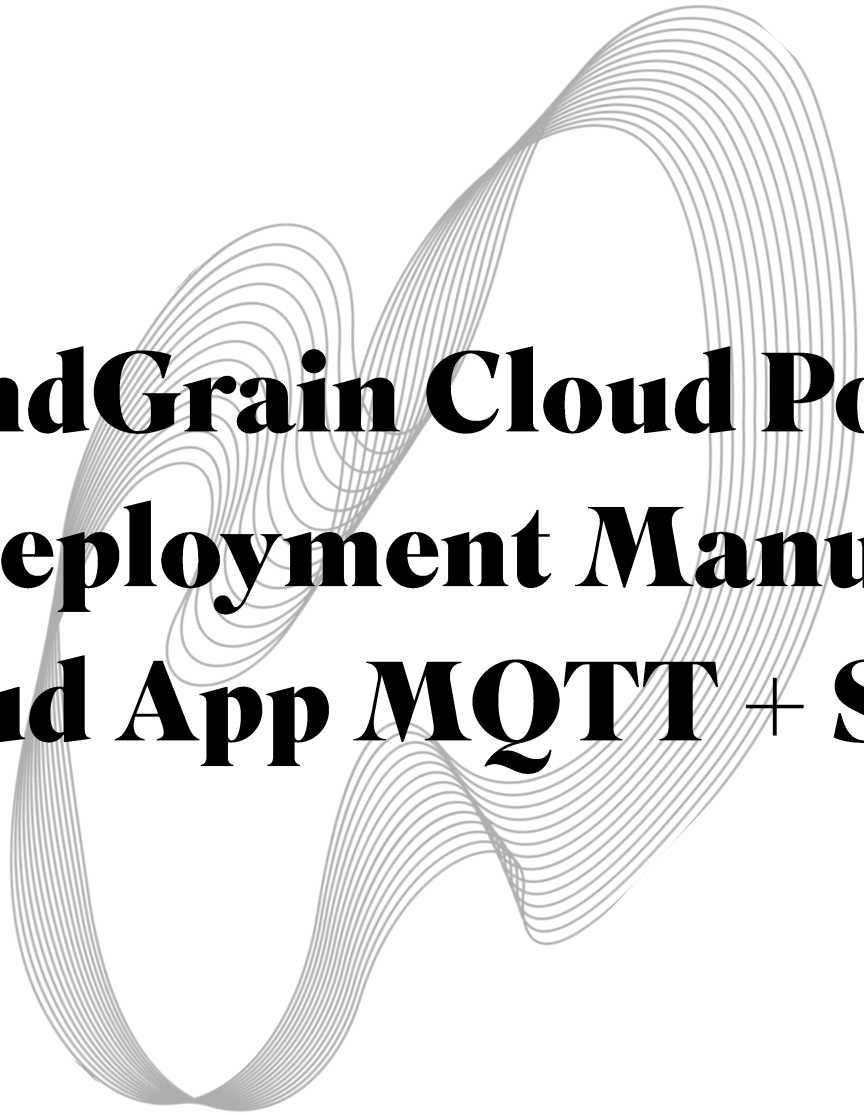
ikerlan

MEMBER OF BASQUE RESEARCH & TECHNOLOGY ALLIANCE



SandGrain Cloud PoC – Deployment Manual (Cloud App MQTT + SDK).

Made by
Denis Stefanescu, Markel Baskaran
30/03/2026

ÍNDICE

Overview 2
Repository Structure 3
Environment Configuration 3
Docker Image Build (Cloud App) 5
Getting started 5
OPTION 1 – Full Local Deployment (Broker + Cloud Application) 5
Step 1 – Configure environment variables 5
Step 2 – Configure device credentials 5
Step 3 — Build and launch containers 6
OPTION 2 – Using an External MQTT Broker 6
Step 1 – Configure environment variables 6
Step 2 – Build and run the cloud application image 6
Expected Runtime Behavior 7
CRC Support 7

OVERVIEW

This repository (sdk_cloud) provides a complete Proof-of-Concept (PoC) environment for implementing and testing **mutual authentication** and **firware verification** between devices and a cloud backend. The environment includes all the necessary components to simulate the full authentication and firmware verification workflow, from message transport to application-level processing and integration with SandGrain CyberRock API. The PoC architecture is composed of three main components:

- **EMQX MQTT Broker**. EMQX acts as the message transport layer that enables communication between devices and the cloud application. It provides a scalable and reliable MQTT infrastructure for publishing and subscribing to authentication requests and responses. In this setup, the broker serves as the central communication hub where devices send authentication requests and where the cloud application listens and responds accordingly.
- **Cloud PoC Application** (application_PoC_MQTT). This component represents the cloud-side application logic responsible for handling device authentication and firmware verification workflows. The application subscribes to relevant MQTT topics and processes incoming device requests.
- **SandGrain Cloud Python SDK** (sandgrain_cloud_sdk). The SDK provides an interface to the SandGrain CyberRock API, allowing the cloud application to easily perform cryptographic and authentication operations required by the protocol.

The cloud application listens for device requests via MQTT and coordinates the authentication process using the SDK. The main operations supported in this PoC include:

- **requestCW**. Initiates the challenge workflow by requesting a cryptographic challenge associated with the device.
- **replyRW.** Sends the appropriate response to a previously generated challenge as part of the authentication sequence.
- **requestRW.** Used in the mutual authentication flow, allowing both the device and the cloud to verify each other's authenticity.
- **fwVerification.** Reports whether a firmware image is valid for a given device and stores the verification result in the cloud for later installation checks.
- **fwInstallation.** Validates a firmware installation request by first confirming that firmware verification was previously approved for the device, and then checking the installation RW against the value generated by CyberRock.

All components can be deployed together using Docker Compose. This containerized setup allows developers to quickly spin up the full environment and test the SandGrain authentication workflow without needing to configure each component manually.

REPOSITORY STRUCTURE

The repository is organized into several directories and configuration files that together provide the full environment required to run the SandGrain Cloud PoC. Each component has a specific role within the architecture and contributes to the deployment and execution of the mutual authentication workflow.

- **application_PoC_MQTT/**

  This directory contains the main runnable cloud service of the Proof-of-Concept environment.

  The application is responsible for:

  - Connecting to the MQTT broker
  - Listening for incoming device authentication and firmware verification requests
  - Processing the messages received from devices
  - Calling the SandGrain Cloud SDK
  - Publishing responses back to the device through MQTT

  In practice, this component acts as the **cloud-side controller** that orchestrates the authentication flow between devices and the SandGrain backend services.

- **sandgrain_cloud_sdk/**

  This directory contains the internal Python SDK package used by the cloud application.

  The SDK provides a set of Python modules that abstract the interaction with the SandGrain Device API, allowing the cloud application to easily perform operations such as:

  - Requesting authentication challenges
  - Sending challenge responses
  - Executing mutual authentication steps

  By encapsulating the API logic in a dedicated SDK, the application code remains cleaner and easier to maintain while promoting modularity and reusability.

Environment Configuration

- **.env (path: application_PoC_MQTT/.env)**

This file contains the environment variables required by the cloud application, including configuration parameters such as:

- MQTT broker address and credentials

- SandGrain Device API base URL
- Cloudflare Access credentials (if enabled)
- Device login credentials JSON path

An example configuration file is provided to simplify the setup process.

To create your local configuration file:

cp .env.example .env

After creating the file, you must edit it and replace the placeholder values with the correct credentials and configuration parameters for your environment.

⚠ **Important: Modify the .env file before starting the services to ensure the application can correctly connect to the required APIs and MQTT bróker.**

- **device_credentials.JSON (path: application_PoC_MQTT/device_credentials.JSON)**

This file contains the device-specific credentials and configuration required to simulate device authentication within the PoC environment.

An example file is included in the repository.

To generate your configuration file:

cp device_credentials_example.json device_credentials.json

Once the file is created, update it with the appropriate credentials and device information needed for the authentication process.

⚠ **Important: Ensure that the credentials defined in this file match the devices registered in the SandGrain platform.**

- **docker-compose.yml**

This file defines the multi-container deployment configuration used to run the PoC environment.

Using Docker Compose, the repository automatically launches:

- The EMQX MQTT Broker, which handles message transport.
- The Cloud PoC Application, which processes authentication requests.

This approach allows the entire environment to be started with a single command, simplifying deployment and testing.

- **Dockerfile**

The Dockerfile defines the build instructions for the cloud application container.

DOCKER IMAGE BUILD (CLOUD APP)

The cloud application is packaged and deployed as a **Docker container**, which is automatically built using the provided Dockerfile. This container encapsulates the runtime environment and all required dependencies needed to execute the Cloud PoC application, ensuring consistent behavior across different development and testing environments.

- Python base: python:3.12
- Installs required dependencies from the `requirements.txt`.
- Installs the Cloud SDK. This SDK enables the cloud application to interact with the SandGrain Device API and execute the authentication workflow.
- Service initialization. It automatically launches the MQTT cloud service, which connects to the MQTT broker and begins listening for device authentication requests.

GETTING STARTED

This section explains how to build and run the PoC environment. The system can be deployed in two different ways:

1. **Full local deployment** – Docker Compose runs both the MQTT broker and the cloud application.
2. **External broker deployment** – Only the cloud application container is executed, connecting to an already existing MQTT broker.

OPTION 1 – Full Local Deployment (Broker + Cloud Application)

Make sure you have installed:

- Docker
- Docker Compose v2 (docker compose)

Step 1 – Configure environment variables

Create the .env file used by the cloud application:

cp application_PoC_MQTT/.env.example application_PoC_MQTT/.env

⚠ **Important:** Replace all placeholder values with valid credentials before starting the services.

Step 2 – Configure device credentials

Create the device credentials configuration file:

cp application_PoC_MQTT/device_credentials_example.json application_PoC_MQTT/device_credentials.json

Edit the file and provide the correct device credentials required for the authentication flow.

Step 3 — Build and launch containers

Run the following command from the repository root:

docker compose up -d –build

This command will build and start the following services:

- **emqx-enterprise** – MQTT broker used for message transport
- **sandgrain-cloud-app** – Cloud PoC application

OPTION 2 – Using an External MQTT Broker

If an MQTT broker (for example EMQX, Mosquitto, or a managed broker) is already available in your infrastructure, you can run **only the cloud application container** and connect it to the external broker.

Step 1 – Configure environment variables

Update the MQTT configuration in:

application_PoC_MQTT/.env

Example:

```
MQTT_HOST=<external-broker-host>
MQTT_PORT=1883
MQTT_USERNAME=<username>
MQTT_PASSWORD=<password>
```

Make sure the broker is reachable from the machine where the container will run.

Step 2 – Build and run the cloud application image

docker build -t sandgrain-cloud-app .

docker run sandgrain-cloud-app

The container will start the MQTT cloud service and connect to the configured external broker.

EXPECTED RUNTIME BEHAVIOR

Once running, the cloud application will:

- Subscribe to wildcard topic: `sandgrain/v1/+/req/+`
- Receive requests from embedded devices
- Dispatch actions through handlers (handlers.py)
- Call SandGrain Device API endpoints via the SDK
- Publish responses back to: `sandgrain/v1/<device_id>/rsp/<action>`

CRC SUPPORT

CRC support is **optional** and **disabled by default**.

The system automatically enables CRC validation when an incoming request includes a CRC field in the frame. In this case, the CRC value of the request is verified before the message is processed.

If the CRC validation succeeds, the response is generated including the CRC of the response payload. This mechanism provides an additional layer of data integrity verification during message exchange.

On the device side, CRC processing is performed only when the CRC field is present in the received frame. If the field is not included, CRC handling is considered disabled and the message is processed normally.